

四种情况一览

最小值 + x 递增: HullMin::query_inc(x)

最小值 + x 递减: HullMin::query_dec(x)

最大值 + x 递增: HullMax::query_inc(x) (内部是最小值壳 + 取反)

最大值 + x 递减: HullMax::query_dec(x)

```

struct Line {
    long long m, b;
    long long f(long long x) const {
        __int128 v = (__int128)m * x + b;
        return (long long)v;
    }
};

static inline bool bad_lower(const Line& a, const Line& b, const Line& c){
    // 下凸壳（求最小值）判 b 冗余: (b-b_a)(m_b-m_c) >= (b_c-b_b)(m_a-m_b)
    __int128 L = (__int128)(b.b - a.b) * (b.m - c.m);
    __int128 R = (__int128)(c.b - b.b) * (a.m - b.m);
    return L >= R;
}

struct HullMin {                // 维护“最小值”的下凸壳
    std::deque<Line> q;
    void add(long long m, long long b){        // 斜率按一个方向单调加入（增或减都行）
        Line c{m,b};
        if (!q.empty() && q.back().m == m) { // 同斜率：保留截距更小的
            if (b >= q.back().b) return;
            q.pop_back();
        }
        while (q.size() >= 2 && bad_lower(q[q.size()-2], q.back(), c)) q.pop_back();
        q.push_back(c);
    }
    long long query_inc(long long x){           // x 递增
        while (q.size() >= 2 && q[0].f(x) >= q[1].f(x)) q.pop_front();
        return q[0].f(x);
    }
    long long query_dec(long long x){           // x 递减
        while (q.size() >= 2 && q.back().f(x) >= q[q.size()-2].f(x)) q.pop_back();
        return q.back().f(x);
    }
};

```

// “最大值”四种情况：用取反技巧复用下凸壳（最稳）

```

struct HullMax {
    HullMin H;                    // 在 H 里存 (-m, -b)
    void add(long long m, long long b){ H.add(-m, -b); }
    long long query_inc(long long x){ return -H.query_inc(x); } // x 递增
};

```

```
long long query_dec(long long x){ return -H.query_dec(x); } // x 递减  
};
```

斜率优化加二分(要求加入的斜率单调)

```

// ===== 核心：最小值 + 斜率严格递减（任意顺序查询，二分） =====
struct LineInt {
    long long m, b;
    long long xLeft;    // 该线在壳中开始生效的最小整数 x
};

static inline long long floor_div128(__int128 a, __int128 b){
    // floor(a / b)    (b 允许为负; C++ / 向零取整需修正)
    if (b < 0) a = -a, b = -b;
    if (a >= 0) return (long long)(a / b);
    return (long long)((a - (b - 1)) / b);
}

static inline long long ceil_div128(__int128 a, __int128 b){
    // ceil(a / b)
    if (b < 0) a = -a, b = -b;
    if (a >= 0) return (long long)((a + (b - 1)) / b);
    return (long long)(a / b);
}

struct CHTMinBinInt {
    // 要求：add 时 m 严格递减
    static constexpr long long NINF = (long long)-4e18;
    std::vector<LineInt> H;

    // 计算“new 从何处起优于 old”的最小整数 x
    static long long firstBetterX(const LineInt& oldL, const LineInt& neuL){
        // 使 neu(x) <= old(x):
        // (m2 - m1) * x <= (b1 - b2)
        __int128 A = (__int128)oldL.b - neuL.b;
        __int128 B = (__int128)neuL.m - oldL.m;    // 对“递减”来说：B < 0
        // x >= ceil(A / B) (B 可能为负也 OK)
        return ceil_div128(A, B);
    }

    void clear(){ H.clear(); }
    int size() const { return (int)H.size(); }

    void add(long long m, long long b){
        LineInt nl{m, b, NINF};
        if (!H.empty() && H.back().m == m){          // 同斜率：保留截距更小（求最小值）
            if (b >= H.back().b) return;
            H.pop_back();
        }
        while (!H.empty()){

```

```

        long long x = firstBetterX(H.back(), nl);
        if (x <= H.back().xLeft) H.pop_back(); // 末尾线完全被新线覆盖
        else { nl.xLeft = x; break; }
    }
    if (H.empty()) nl.xLeft = NINF;
    H.push_back(nl);
}

long long query(long long x) const { // 任意顺序查询
    // 找到 xLeft <= x 的最后一条线
    int l = 0, r = (int)H.size() - 1, ans = r;
    while (l <= r){
        int mid = (l + r) >> 1;
        if (H[mid].xLeft <= x){ ans = mid; l = mid + 1; }
        else r = mid - 1;
    }
    __int128 v = (__int128)H[ans].m * x + H[ans].b;
    return (long long)v;
}

};

// ===== 四种封装：统一接口 add(m,b) / query(x) =====
// 1) 最小值 + 斜率递减（本体）
using CHTMinSlopeDec = CHTMinBinInt;

// 2) 最小值 + 斜率递增（映射：m'=-m, x'=-x）
struct CHTMinSlopeInc {
    CHTMinBinInt base;
    void add(long long m, long long b){ base.add(-m, b); }
    long long query(long long x) const { return base.query(-x); }
    void clear(){ base.clear(); }
    int size() const { return base.size(); }
};

// 3) 最大值 + 斜率递减
// max(m*x+b) = - min(m*(-x) + (-b)) → 传 (m, -b), 查 -x, 再取反
struct CHTMaxSlopeDec {
    CHTMinBinInt base;
    void add(long long m, long long b){ base.add(m, -b); }
    long long query(long long x) const { return -base.query(-x); }
    void clear(){ base.clear(); }
    int size() const { return base.size(); }
};

```

```

// 4) 最大值 + 斜率递增
//     $\max(m*x+b) = -\min((-m)*x + (-b)) \rightarrow$  传  $(-m, -b)$ , 查  $x$ , 取反
struct CHTMaxSlopeInc {
    CHTMinBinInt base;
    void add(long long m, long long b){ base.add(-m, -b); }
    long long query(long long x) const { return -base.query(x); }
    void clear(){ base.clear(); }
    int size() const { return base.size(); }
};

// long double 实现

// struct CHTMinBinLD { // 最小值 + 斜率严格递减
//     struct Line {
//         long long m, b;
//         long double xLeft; // 该线在壳中开始生效的 x (断点)
//     };

//     static inline long double INF() {
//         return std::numeric_limits<long double>::infinity();
//     }

//     // 交点:  $a(x) == b(x)$  的  $x$ , 右侧由  $b$  更优 (斜率递减:  $a.m > b.m$ )
//     static inline long double intersect(const Line& a, const Line& b) {
//         // 分母  $b.m - a.m < 0$ ; 用 long double 防精度
//         return (long double)(a.b - b.b) / (long double)(b.m - a.m);
//     }

//     std::vector<Line> H; // xLeft 单调递增

//     void clear() { H.clear(); }
//     int size() const { return (int)H.size(); }

//     void add(long long m, long long b) {
//         Line nl{m, b, -INF()};
//         if (!H.empty() && H.back().m == m) {
//             // 同斜率: 求最小值保留截距更小的
//             if (b >= H.back().b) return;
//             H.pop_back();
//         }
//         while (!H.empty()) {
//             long double x = intersect(H.back(), nl);

```

```

//          if (x <= H.back().xLeft) H.pop_back(); // 末尾线被完全覆盖
//          else { nl.xLeft = x; break; }
//      }
//      if (H.empty()) nl.xLeft = -INF();
//      H.push_back(nl);
//  }

//  // 整数 x 查询, 返回 long long (内部用 128 位避免溢出)
//  long long query(long long x) const {
//      // 要求: 调用前 H 非空
//      long double X = (long double)x;
//      int l = 0, r = (int)H.size() - 1, ans = r;
//      while (l <= r) {
//          int mid = (l + r) >> 1;
//          if (H[mid].xLeft <= X) { ans = mid; l = mid + 1; }
//          else r = mid - 1;
//      }
//      __int128 v = (__int128)H[ans].m * x + H[ans].b;
//      return (long long)v;
//  }

//  // 浮点 x 查询, 返回 long double (可选)
//  long double query_ld(long double x) const {
//      int l = 0, r = (int)H.size() - 1, ans = r;
//      while (l <= r) {
//          int mid = (l + r) >> 1;
//          if (H[mid].xLeft <= x) { ans = mid; l = mid + 1; }
//          else r = mid - 1;
//      }
//      return (long double)H[ans].m * x + (long double)H[ans].b;
//  }
// };

```